

```
# =====  
# 0. 기존 데이터 삭제  
# =====
```

```
# 외래키 체크 해제  
SET FOREIGN_KEY_CHECKS = 0;
```

```
# 모든 테이블 데이터 삭제 (순서 상관 없음)  
TRUNCATE TABLE reviews;  
TRUNCATE TABLE cart;  
TRUNCATE TABLE order_alert;  
TRUNCATE TABLE payments;  
TRUNCATE TABLE order_items;  
TRUNCATE TABLE orders;  
TRUNCATE TABLE products;  
TRUNCATE TABLE users;  
TRUNCATE TABLE category;
```

```
# 외래키 체크 다시 활성화  
SET FOREIGN_KEY_CHECKS = 1;
```

```
# =====  
# 1. 기초 데이터 (카테고리 & 회원) 생성  
# =====
```

```
DELIMITER $$
```

```
# 기존 프로시저 삭제  
DROP PROCEDURE IF EXISTS loop_insert_users_realistic$$
```

```
# 수정된 프로시저 생성  
CREATE PROCEDURE loop_insert_users_realistic()  
BEGIN
```

```
    DECLARE i INT DEFAULT 1;  
    DECLARE v_last_name VARCHAR(10);  
    DECLARE v_first_name VARCHAR(10);  
    DECLARE v_full_name VARCHAR(20);  
    DECLARE v_domain VARCHAR(20);  
    DECLARE v_gu VARCHAR(10);  
    DECLARE v_dong VARCHAR(10);  
    DECLARE v_rand_num INT;
```

```
    SET autocommit = 0; # 속도 향상
```

```
    WHILE i <= 10000 DO
```

```
        # 이름 개수(19개)에 맞춰 랜덤 범위 수정 (20 -> 19)  
        SET v_last_name = ELT(FLOOR(1 + RAND() * 10), '김', '이', '박', '최', '정', '강', '조', '윤', '장', '임');  
        SET v_first_name = ELT(FLOOR(1 + RAND() * 19), '민준', '서준', '도윤', '예준', '시우', '하준', '지호',  
'지유', '서윤', '서연', '민서', '하은', '지민', '수아', '하윤', '지우', '민재', '현수', '예은');  
        SET v_full_name = CONCAT(v_last_name, v_first_name);
```

```
        # 주소 조합
```

```

SET v_gu = ELT(FLOOR(1 + RAND() * 10), '강남구', '서초구', '송파구', '마포구', '용산구', '영등포구',
'성동구', '종로구', '강서구', '노원구');
SET v_dong = ELT(FLOOR(1 + RAND() * 8), '역삼동', '반포동', '잠실동', '연남동', '한남동', '여의도동',
'성수동', '평창동');
SET v_rand_num = FLOOR(1 + RAND() * 100);

INSERT INTO users (name, email, password, tel, address, point)
VALUES (
    v_full_name,
    CONCAT(SUBSTRING(MD5(RAND()), 1, 8), i, '@', ELT(FLOOR(1 + RAND() * 5), 'naver.com',
'gmail.com', 'daum.net', 'kakao.com', 'outlook.com')),
    SHA2('password1234', 256), # 비밀번호 암호화
    CONCAT('010-', LPAD(FLOOR(RAND()*9000)+1000, 4, '0'), '-', LPAD(FLOOR(RAND()*9000)+1000,
4, '0')),
    CONCAT('서울시 ', v_gu, ' ', v_dong, ' ', v_rand_num, '번지'),
    FLOOR(RAND() * 50000)
);
SET i = i + 1;
IF i % 1000 = 0 THEN COMMIT; END IF;
END WHILE;
COMMIT;
SET autocommit = 1;
END$$

```

DELIMITER ;

```

# =====
# 2. 상품 데이터 삽입
# =====

```

DELIMITER \$\$

```

DROP PROCEDURE IF EXISTS loop_insert_products_realistic$$
CREATE PROCEDURE loop_insert_products_realistic()
BEGIN
    DECLARE i INT DEFAULT 1;
    DECLARE v_cat_idx INT;
    DECLARE v_name VARCHAR(100);
    DECLARE v_adj VARCHAR(20);
    DECLARE v_noun VARCHAR(20);

    SET autocommit = 0;

    WHILE i <= 2000 DO
        SET v_cat_idx = FLOOR(1 + RAND() * 10); # 카테고리 1~10 랜덤

        # 카테고리별로 상품명 조합 로직 (CASE문 활용)
        CASE v_cat_idx
            WHEN 1 THEN # 식품
                SET v_adj = ELT(FLOOR(1 + RAND() * 5), '신선한', '유기농', '달콤한', '프리미엄', '제주산');
                SET v_noun = ELT(FLOOR(1 + RAND() * 5), '우유 1L', '삼겹살 500g', '사과 1박스', '쌀 10kg',
'생수 2L');
            WHEN 2 THEN # 생활용품

```

```

        SET v_adj = ELT(FLOOR(1 + RAND() * 5), '부드러운', '강력한', '오래가는', '순면', '대용량');
        SET v_noun = ELT(FLOOR(1 + RAND() * 5), '화장지 30롤', '세탁세제', '섬유유연제', '물티슈',
'치약 세트');
        WHEN 5 THEN # 가전
            SET v_adj = ELT(FLOOR(1 + RAND() * 5), '최신형', '초고속', '스마트', '무선', '저소음');
            SET v_noun = ELT(FLOOR(1 + RAND() * 5), '무선 청소기', '공기청정기', '블루투스 스피커', '헤어
드라이어', '가습기');
            ELSE # 그 외 카테고리 (공통)
                SET v_adj = ELT(FLOOR(1 + RAND() * 5), '인기만점', '가성비', '럭셔리', '필수템', '한정판');
                SET v_noun = CONCAT('상품_', i); # 나머지는 간단하게
            END CASE;

        SET v_name = CONCAT(v_adj, ' ', v_noun);

        INSERT INTO products (category_idx, name, price, stock, description)
        VALUES (
            v_cat_idx,
            v_name,
            FLOOR(1000 + RAND() * 99000), # 1000원 ~ 10만원
            FLOOR(10 + RAND() * 500), # 재고 10 ~ 500
            CONCAT('이 상품은 ', v_name, '입니다. 최고의 품질을 보장합니다.')
        );
        SET i = i + 1;
        IF i % 500 = 0 THEN COMMIT; END IF;
    END WHILE;
    COMMIT;
    SET autocommit = 1;
END$$

DELIMITER ;

# =====
# 3. 주문 & 결제 데이터 (10만 건) 생성
# =====

DELIMITER $$

DROP PROCEDURE IF EXISTS loop_insert_orders_transaction$$

CREATE PROCEDURE loop_insert_orders_transaction()
BEGIN
    DECLARE i INT DEFAULT 1;
    DECLARE v_user_idx INT;
    DECLARE v_order_idx INT;
    DECLARE v_total INT;
    DECLARE v_rand_date TIMESTAMP;

    SET autocommit = 0;

    WHILE i <= 100000 DO
        SET v_user_idx = FLOOR(1 + RAND() * 10000); # 유저 1만명 중 랜덤
        SET v_rand_date = DATE_SUB(NOW(), INTERVAL FLOOR(RAND() * 365) DAY); # 1년 내 랜덤 날짜
    
```

#### # 1. 주문 생성

```
INSERT INTO orders (users_idx, total_price, status, created_at)
VALUES (v_user_idx, 0, ELT(FLOOR(1+RAND()*3), '구매확정', '배송완료', '결제완료'), v_rand_date);

SET v_order_idx = LAST_INSERT_ID();
```

#### # 2. 주문 상세 생성

```
INSERT INTO order_items (orders_idx, products_idx, `count`, price_snapshot, products_status)
SELECT
    v_order_idx,
    products_idx,
    FLOOR(1 + RAND() * 3),
    price,
    '구매확정'
FROM products
ORDER BY RAND() LIMIT 1;
```

#### # 3. 총액 업데이트

```
SELECT SUM(`count` * price_snapshot) INTO v_total
FROM order_items WHERE orders_idx = v_order_idx;
```

```
UPDATE orders SET total_price = v_total WHERE orders_idx = v_order_idx;
```

#### # 4. 결제 정보 생성

```
INSERT INTO payments (orders_idx, method, real_amount, created_at)
VALUES (v_order_idx, ELT(FLOOR(1+RAND()*3), 'CARD', 'KAKAO', 'NAVER'), v_total, v_rand_date);
```

```
SET i = i + 1;
IF i % 1000 = 0 THEN COMMIT; END IF;
END WHILE;
COMMIT;
SET autocommit = 1;
```

END\$\$

DELIMITER ;

# =====

#### # 4. 알림, 장바구니, 리뷰 (부가 데이터) 삽입

# =====

DELIMITER \$\$

#### # 1. 구매 알림 (3만 건)

```
DROP PROCEDURE IF EXISTS loop_insert_alerts$$
CREATE PROCEDURE loop_insert_alerts()
BEGIN
```

```
    DECLARE i INT DEFAULT 1;
```

```
    SET autocommit = 0;
```

```
    WHILE i <= 30000 DO
```

```
        INSERT INTO order_alert (users_idx, products_idx, con_cycle_set, con_cycle, next_alert_date,
alert_set)
```

```
        VALUES (
            FLOOR(1 + RAND() * 10000), # 회원 랜덤
```

```

FLOOR(1 + RAND() * 2000), # 상품 랜덤 (2000개 기준)
IF(RAND() > 0.5, 1, 0), # 수동/자동 반반
30, # 주기
DATE_ADD(NOW(), INTERVAL FLOOR(RAND() * 20) - 10 DAY), # 오늘 전후 10일
1 # 알림 ON
);
SET i = i + 1;
IF i % 1000 = 0 THEN COMMIT; END IF;
END WHILE;
COMMIT;
SET autocommit = 1;
END$$

# 2. 리뷰 데이터 (5천 건)
DROP PROCEDURE IF EXISTS loop_insert_reviews$$
CREATE PROCEDURE loop_insert_reviews()
BEGIN
    DECLARE i INT DEFAULT 1;
    DECLARE v_content VARCHAR(100);
    SET autocommit = 0;
    WHILE i <= 5000 DO
        SET v_content = ELT(FLOOR(1 + RAND() * 5), '배송이 정말 빨라요!', '재구매 의사 있습니다.', '가격
대비 품질이 좋네요.', '생각보다 별로였어요.', '잘 쓰겠습니다~');

        # reviews
        INSERT INTO reviews (users_idx, products_idx, reviews_content)
        VALUES (
            FLOOR(1 + RAND() * 10000),
            FLOOR(1 + RAND() * 2000),
            v_content
        );
        SET i = i + 1;
        IF i % 1000 = 0 THEN COMMIT; END IF;
    END WHILE;
    COMMIT;
    SET autocommit = 1;
END$$

DELIMITER ;

# =====
# 5. 장바구니 데이터 생성
# =====

DELIMITER $$

DROP PROCEDURE IF EXISTS loop_insert_cart$$

CREATE PROCEDURE loop_insert_cart()
BEGIN
    DECLARE i INT DEFAULT 1;

    SET autocommit = 0; # 속도 향상을 위해 오토커밋 해제

```

```

# 5,000개의 장바구니 아이템 생성
WHILE i <= 5000 DO
    INSERT INTO cart (users_idx, products_idx, `count`, created_at)
    VALUES (
        FLOOR(1 + RAND() * 10000), # 회원: 1 ~ 10000번 중 랜덤
        FLOOR(1 + RAND() * 2000), # 상품: 1 ~ 2000번 중 랜덤
        FLOOR(1 + RAND() * 5),    # 수량: 1 ~ 5개 랜덤
        DATE_SUB(NOW(), INTERVAL FLOOR(RAND() * 30) DAY) # 날짜: 최근 30일 이내 랜덤
    );

    SET i = i + 1;

    # 1000건마다 커밋 (부하 방지)
    IF i % 1000 = 0 THEN
        COMMIT;
    END IF;
END WHILE;

COMMIT;
SET autocommit = 1;
END$$

DELIMITER ;

# =====
# 6. 데이터 실행
# =====

CALL loop_insert_categories();
CALL loop_insert_users_realistic(); # 1만명 생성
CALL loop_insert_products_realistic(); # 리얼한 상품명 2천개
CALL loop_insert_orders_transaction(); # 10만건 주문 (시간 좀 걸림)
CALL loop_insert_alerts();
CALL loop_insert_reviews();
CALL loop_insert_cart();

# 확인
SELECT '회원' AS TBL, COUNT(*) FROM users
UNION ALL SELECT '상품', COUNT(*) FROM products
UNION ALL SELECT '주문', COUNT(*) FROM orders;

# =====
# 7. 5,000명 대규모 재구매 패턴 생성성
# =====

DELIMITER $$

DROP PROCEDURE IF EXISTS insert_massive_reorder_patterns$$
CREATE PROCEDURE insert_massive_reorder_patterns()
BEGIN
    DECLARE i INT DEFAULT 1;      # 유저 카운트 (1번부터 5000번까지)
    DECLARE j INT DEFAULT 0;      # 구매 횟수 카운트 (0~3회)

```

```
DECLARE v_user_idx INT;
DECLARE v_prod_idx INT;
DECLARE v_order_idx INT;
DECLARE v_base_date TIMESTAMP;
DECLARE v_price INT;
```

```
SET autocommit = 0; # 대량 입력을 위한 자동 커밋 해제
```

```
# 1. 상위 5,000명의 유저를 대상으로 반복
```

```
WHILE i <= 5000 DO
```

```
    SET v_user_idx = i;
```

```
    # 각 유저마다 재구매할 상품 1종을 랜덤하게 선정
```

```
    SET v_prod_idx = FLOOR(1 + RAND() * 2000);
```

```
    # 해당 상품의 가격 조회
```

```
    SELECT price INTO v_price FROM products WHERE products_idx = v_prod_idx;
```

```
    # 기준 날짜: 오늘로부터 약 110일 전 (4회 구매 시 마지막 구매가 오늘 근처가 되도록)
```

```
    SET v_base_date = DATE_SUB(NOW(), INTERVAL 110 DAY);
```

```
# 2. 30일 간격으로 4회 구매 루프
```

```
SET j = 0;
```

```
WHILE j < 4 DO
```

```
    # 주문 생성 (날짜에 약간의 랜덤성을 주어 자연스럽게 만듦)
```

```
    INSERT INTO orders (users_idx, total_price, status, created_at)
```

```
    VALUES (
```

```
        v_user_idx,
```

```
        v_price,
```

```
        '구매확정',
```

```
        DATE_ADD(v_base_date, INTERVAL (j * 30) + FLOOR(RAND() * 5) DAY)
```

```
    );
```

```
    SET v_order_idx = LAST_INSERT_ID();
```

```
    # 주문 상세 기록
```

```
    INSERT INTO order_items (orders_idx, products_idx, `count`, price_snapshot, products_status)
```

```
    VALUES (v_order_idx, v_prod_idx, 1, v_price, '구매확정');
```

```
    # 결제 기록
```

```
    INSERT INTO payments (orders_idx, method, real_amount, created_at)
```

```
    VALUES (
```

```
        v_order_idx,
```

```
        ELT(FLOOR(1+RAND()*3), 'CARD', 'KAKAO', 'NAVER'),
```

```
        v_price,
```

```
        DATE_ADD(v_base_date, INTERVAL (j * 30) DAY)
```

```
    );
```

```
    SET j = j + 1;
```

```
END WHILE;
```

```
SET i = i + 1;
```

```
# 500명 단위로 중간 커밋 (메모리 및 로그 부하 방지)
IF i % 500 = 0 THEN
    COMMIT;
END IF;
END WHILE;

COMMIT;
SET autocommit = 1;
END$$

DELIMITER ;

# 프로시저 실행
CALL insert_massive_reorder_patterns();
```